

Pandera or Pydantic?

Drawing the validation boundary in Cordwell's pipeline

Continues the Cordwell Home & Hardware scenario · Draws on Modules 01, 03–05, and 08

How this works. Same teams, same roles, **~45 minutes**, then we reconvene and each team reports back. Still no coding — whiteboard and argue. Both of these libraries let you write *rating must be between 1 and 5* in about the same number of characters. That surface similarity is exactly why engineers reach for the wrong one. Your job today is to find the line between them and be able to defend where you drew it.

Where we left off: in Exercise 1 your team planned the extraction and produced a filtered DataFrame of candidate training examples for Cordwell's support assistant. Everything downstream now rests on a claim nobody has proved — that this data is actually fit to ship.

THE ONE IDEA — THREE AXES

Both libraries declare a schema and check data against it. They differ on three things, and every other difference follows from these:

	Pydantic	Pandera
Unit of work	One record	One DataFrame (columns + whole table)
Can it see the other rows?	No — never	Yes — that's the point
What comes back	A new, typed object (it parses and coerces)	The same DataFrame (it inspects in place)

The diagnostic question. Before choosing a tool, ask: *“Could I answer this by looking at exactly one record, in isolation?”*

- **Yes** → it's a record-level contract. That belongs at a boundary. Pydantic.
- **No** → it's a dataset-level invariant. Only Pandera can even express it.

Rule of thumb: Boundaries → Pydantic. Batches → Pandera.

Part 1 Why they look identical (name the confusion)

Both snippets below run cleanly on our stack. Read them before you argue.

Pandera — DataFrameModel

```
import pandera.pandas as pa
from pandera.typing import Series

class TicketSchema(pa.DataFrameModel):
    ticket_id: Series[int] = pa.Field(
        unique=True)
    rating: Series[int] = pa.Field(
        ge=1, le=5)
    category: Series[str] = pa.Field(
        isin=["order_status", "returns"])

out = TicketSchema.validate(df)
# -> out is a DataFrame
```

Pydantic — BaseModel

```
from pydantic import BaseModel, Field
from typing import Literal

class Ticket(BaseModel):
    ticket_id: int

    rating: int = Field(
        ge=1, le=5)
    category: Literal[
        "order_status", "returns"]

t = Ticket(**record)
# -> t is a Ticket
```

1. Circle every genuine difference you can find. Two are easy to miss: the word `Series`, and what each `.validate()` / constructor hands back. Why do those two details matter more than they look?
2. `pa.Field(ge=1, le=5)` and `Field(ge=1, le=5)` express exactly the same constraint. If the constraint is identical, why does it matter which library you wrote it in? What has actually changed?
3. Only the left-hand schema can enforce `unique=True` on `ticket_id`. Don't answer “the other one doesn't support it.” Answer: what would Pydantic have to know that it structurally cannot know?

Part 2 Map the tools onto Cordwell's pipeline

For each stage, decide: *Pandera*, *Pydantic*, or *genuinely both*? Fill the table in, then defend the two rows you argued about longest.

Stage or check	Which tool?	Why — one sentence
The <code>filter_config</code> thresholds your team read from a YAML file		
Each JSON record streaming back from the paginated reviews API		
A 100k-row chunk just returned by <code>pd.read_sql_query(..., chunksize=...)</code>		
“No single category exceeds 20% of the dataset” (Module 01's balance threshold)		
“ <code>reply_text</code> is never null and is 20–2000 characters”, on the filtered DataFrame		
Each line of the final messages-format JSONL, as it is written out		
The FastAPI endpoint that will serve this assistant in Week 8		

4. At least two rows are honestly “both.” Which ones, and what does each tool contribute that the other doesn't?
5. Notice that the two rows involving an API — one inbound, one outbound — landed on the same tool. What do a REST response and a line of JSONL have in common that a DataFrame chunk doesn't?

Part 3 Run the diagnostic question

Sort each check below into *record-level* or *dataset-level*, then name the tool. One of them is trickier than it looks.

- (a) rating is between 1 and 5
- (b) ticket_id is unique across the batch
- (c) price parses from the string "19.99" into a float
- (d) no category exceeds 20% of the dataset
- (e) reply_text is never null
- (f) resolved_at is later than created_at, within the same ticket
- (g) the duplication rate across the dataset is under 2%

- Which of (a)–(g) can each tool express? Item (f) should start an argument — settle it. Does “cross-field” mean the same thing as “cross-row”?
- The manufacturer spec API returns {"sku": "CW-1003", "price": "not-a-price"}. Your teammate pushes the raw response list straight into a DataFrame. What dtype does the price column end up with, and what does a downstream Pandera Column(float) check do about it? What should have happened first?
- Pandera cannot validate a raw list of dictionaries at all — it raises before it checks anything. Why not? What does it fundamentally require before it has an opinion, and what does that tell you about where it can and cannot sit in the pipeline?

Part 4 Spot the defect

This one is real. Two slides from this week disagree with each other, and the Pydantic model is the one that's wrong.

What Module 01 actually writes to JSONL

```
example = {
  "messages": [
    {"role": "system",
     "content": "You are a ..."},
    {"role": "user",
     "content": row['feedback_text']},
    {"role": "assistant",
     "content": "Rating: 5/5 ..."},
  ]
}
```

What Module 08's Pydantic model validates

```
class TrainingExample(BaseModel):
    prompt: str = Field(
        min_length=1, max_length=1000)
    completion: str = Field(
        min_length=1, max_length=2000)
    metadata: dict
    created_at: datetime
    quality_score: float = Field(
        ge=0.0, le=1.0)
```

- Point the model on the right at a line produced by the code on the left. What happens, and which field does it complain about first?
- Which of these two shapes do current chat fine-tuning endpoints actually accept? What does that make the model on the right — a bug, or merely out of date? Does the distinction matter to the engineer who ships it?
- On the whiteboard, sketch the Pydantic model that would genuinely guard one line of messages-format JSONL. You'll need more than one class. What are the nested pieces, and which field should be constrained to a fixed set of values?
- Deeper: that model would have validated cleanly against a file it was never designed for, had the field names happened to match. What does that tell you about writing a schema before you have pinned down the contract it's supposed to enforce?

Part 5 Trade-offs and a prediction (stretch)

Pick what you have time for. The first one is a wager — write your number down before your instructor reveals the measured answer.

13. Predict. You have a 500,000-row DataFrame already in memory and you want to check three column constraints. How many times faster is a Pandera schema than looping an equivalent Pydantic model over `df.to_dict("records")`? Commit to a number as a team.
14. Now the twist: Pydantic validates a single record in roughly 1.3 microseconds — it is genuinely fast. So why is looping it over a DataFrame still the wrong call, while using it on records streaming in from an API, one page at a time, is exactly right?
15. A teammate proposes: “Just use Pydantic everywhere. One tool, less to learn, and it validates our JSONL anyway.” Make the strongest case for them. Then make the strongest case against. Which checks from Part 3 would Cordwell simply lose?
16. Module 08 offers four failure strategies: fail fast, log and skip, quarantine, auto-correct. Pick one for the reviews API and a different one for the final JSONL write. Justify why the same pipeline warrants different answers at different boundaries.

Report back to the group

Your reporter has ~3 minutes. Come ready with:

- **The one-sentence rule** your team would put in Cordwell's engineering wiki for choosing between these two tools.
- **One check** in your pipeline that Pydantic structurally cannot express — and the reason, not just the fact.
- **One place** your team first reached for the wrong tool — and what tipped you off.
- **Your prediction** from Part 5, question 1. Say the number out loud before the reveal.

Cordwell Home & Hardware is a fictional company; all data described is synthetic and for training use only.